ISA specification: 8-bit data, 16-bit instructions

Eight general registers R0-R7 (none hardwired). PC is an 8-bit word address into a 256 x 16 ROM, starts at 0x00 and normally increments by one. Arithmetic wraps modulo 256. Branch/jump targets are absolute 8-bit word addresses, no delay slots. Field naming: **A-field** = bits [11:9] (destination Rd, and the register read on read-port A); **B-field** = bits [8:6] (register read on read-port B). The decoder never looks at reserved bits; the assembler emits them as 0.

1. Instruction formats (three)

Format	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	op				Rd (A)			Rs (B)			0 (rsvd)					
I	op				Rd/Rs (A)			0	imm8 / target							
J	op				0 (rsvd)			target								

2. Opcode map and decode table

Op	Mnemonic	Fmt	Semantics	reads A	reads B	writes Rd	ALU / note	Example	Hex
0x0	NOP	R	no state change	-	-	-	all fields ignored	NOP	0000
0x1	LDI Rd, imm8	I	Rd <- imm8 (bit pattern, no extension)	-	-	yes	pass Y=imm	LDI R1, 0x64	1264
0x2	MOV Rd, Rs	R	Rd <- Rs	-	yes	yes	pass Y=B	MOV R1, R3	22C0
0x3	ADD Rd, Rs	R	Rd <- Rd + Rs (mod 256)	yes	yes	yes	x+y	ADD R3, R1	3640
0x4	SUB Rd, Rs	R	Rd <- Rd - Rs (mod 256)	yes	yes	yes	x+~y+1	SUB R1, R1	4240
0x5	AND Rd, Rs	R	Rd <- Rd AND Rs	yes	yes	yes	x AND y	AND R2, R1	5440
0x6	LD Rd, [Rs]	R	Rd <- DATA[Rs]	-	yes	yes	address = B	LD R1, [R7]	63C0
0x7	ST Rd, [Rs]	R	DATA[Rs] <- Rd	yes (data)	yes (addr)	-	data = A, addr = B	ST R2, [R5]	7540
0x8	ADDI Rd, imm8	I	Rd <- Rd + imm8 (mod 256; 0xFF = -1)	yes	-	yes	x+imm	ADDI R2, 0xF4	84F4
0x9	BEQZ Rs, target	I	if Rs = 0: PC <- target	yes	-	-	zero test on A	BEQZ R1, 0x0C	920C
0xA	BNEZ Rs, target	I	if Rs != 0: PC <- target	yes	-	-	zero test on A	BNEZ R1, 0x0C	A20C
0xB	JMP target	J	PC <- target	-	-	-	[11:8] reserved	JMP 0x2A	B02A
0xC	SLTU Rd, Rs (custom)	R	Rd <- (Rd < Rs unsigned) ? 1 : 0	yes	yes	yes	{7'b0, ~carry(x+~y+1)}	SLTU R1, R4	C300
0xD	(reserved)	-	decodes to no control bits: NOP	-	-	-	no write, no stall		D000
0xE	(reserved)	-	decodes to no control bits: NOP	-	-	-	no write, no stall		E000
0xF	HALT	R	stop after older instrs; kill younger	-	-	-	acts in EX	HALT	F000

Decoded control word (control_dec, one-hot 4-to-16 opcode decoder + OR gates): regw = LDI|MOV|ADD|SUB|AND|LD|ADDI|SLTU; aluimm = LDI|ADDI (Y = imm8 instead of B); aluop[1:0] = 00 sum, 01 AND, 10 pass-Y, 11 SLTU (aluop1 = LDI|MOV|SLTU, aluop0 = AND|SLTU); alusub = SUB|SLTU; memr = LD; memw = ST; beqz, bnez, jmp, halt; uses_a = ADD|SUB|AND|ST|ADDI|BEQZ|BNEZ|SLTU; uses_b = MOV|ADD|SUB|AND|LD|ST|SLTU. uses_a/uses_b are only for load-use detection, so fields an instruction does not read (the Rd of LDI/MOV/LD, the B-field of I/J formats, the reserved bits) never cause a stall. Unassigned opcodes 0xD/0xE set no control bit, so they cannot write registers, RAM or I/O, cannot branch and never stall: deterministic NOP behaviour.

3. Immediate and branch semantics

imm8 = IR[7:0] is carried unchanged into ID/EX. LDI copies the pattern; ADDI adds it modulo 256 (0xFF behaves as -1, 0xF4 as -12). Branches and JMP use the same 8 bits as an absolute target; the condition (A-field register, after forwarding, compared with 0 by zero8) and the redirect are resolved in EX. A taken branch loads PC with the target and invalidates IF/ID and ID/EX, i.e. exactly two wrong-path instructions are discarded when nothing is stalled. A not-taken branch costs nothing.

4. Application-specific instruction: SLTU Rd, Rs (opcode 0xC, R-format)

Semantics. Rd <- 0x01 if Rd < Rs as unsigned 8-bit numbers, else 0x00. **Encoding.** 1100 ddd sss 000000, e.g. SLTU R1, R4 = 0xC300. **Datapath change.** None outside the ALU: SLTU reuses the subtract path (alusub = 1 gives x + ~y + 1, whose carry-out is 1 exactly when x >= y). The ALU's 4:1 result mux gets a fourth input {7'b0, NOT cout}, selected by aluop = 11. **Pipeline timing.** Identical to ADD/SUB: operands needed at the start of EX, result available at the end of EX and forwardable from EX/MEM in the next cycle; written in WB. **Hazards.** Reads both A and B fields (uses_a = uses_b = 1), so a load producing either operand right before it causes the standard single bubble (as in every fill/drain loop: LD R1,[R7] ; SLTU R1,R4). Its result feeds BNEZ through EX/MEM forwarding with no stall.

Control-program benefit. The ISA has no flags and no carry/sign bit visible to software, so an unsigned >= test would otherwise need a bit-serial software routine (compare MSBs with AND masks, branch, repeat for 8 bits; about 30-50 instructions per test). With SLTU the fill loop is three instructions (LD level ; SLTU level, target ; BNEZ loop) and exits on *any* sample >= target, so a reading that jumps past the target (99 -> 102) still stops the valve. The drain test uses the operands the other way round (MOV R1, thresh ; SLTU R1, level ; BNEZ loop) to exit on any level <= threshold. An equality test (SUB + BEQZ) would miss skipped thresholds.